

Les bases de la programmation graphique

Java

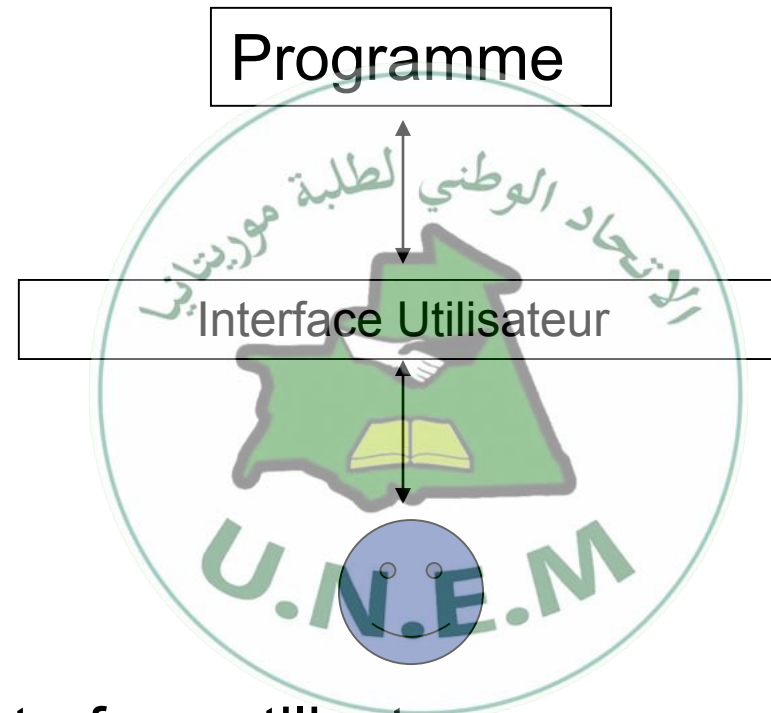


AWT

The Abstract Windowing Toolkit and Swing



Généralité



Rôles d'une interface utilisateur:

- montrer le résultat de l'exécution
 - permettre à l'utilisateur d'interagir
- (1) Montrer – (2) Réagir



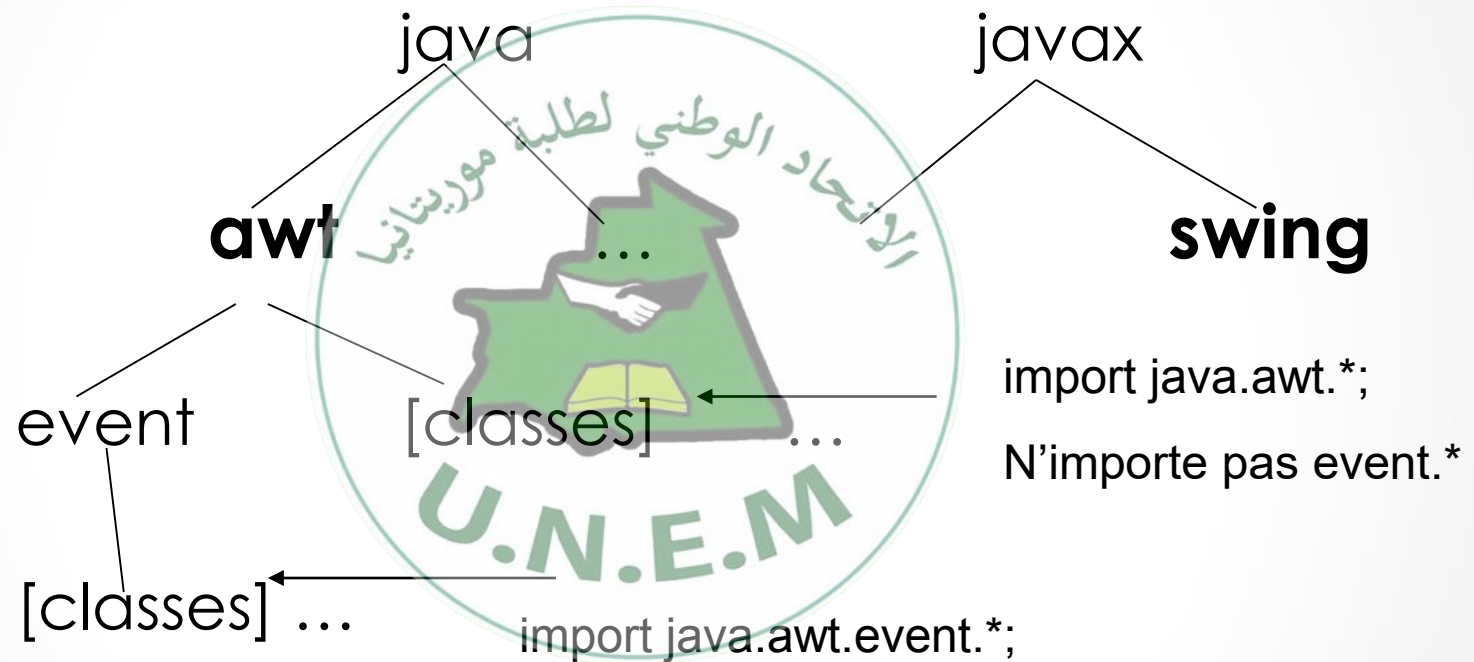
Afficher une interface

- Importer le package (les classes)
 - Les classes sont regroupées en package
 - Importer un package = importer toutes les classes du package
 - `import javax.swing.*;`
- Créer une fenêtre graphique (JFrame, ...)
- Définir les paramètres (taille, ...)
- Afficher
- Différence:
 - `import java.awt.*;` les classes dans awt
 - `import java.awt.event.*;` les classes dans event
 - `import javax.swing.*;`

awt: Abstract Windows Toolkit



Hiérarchie de package

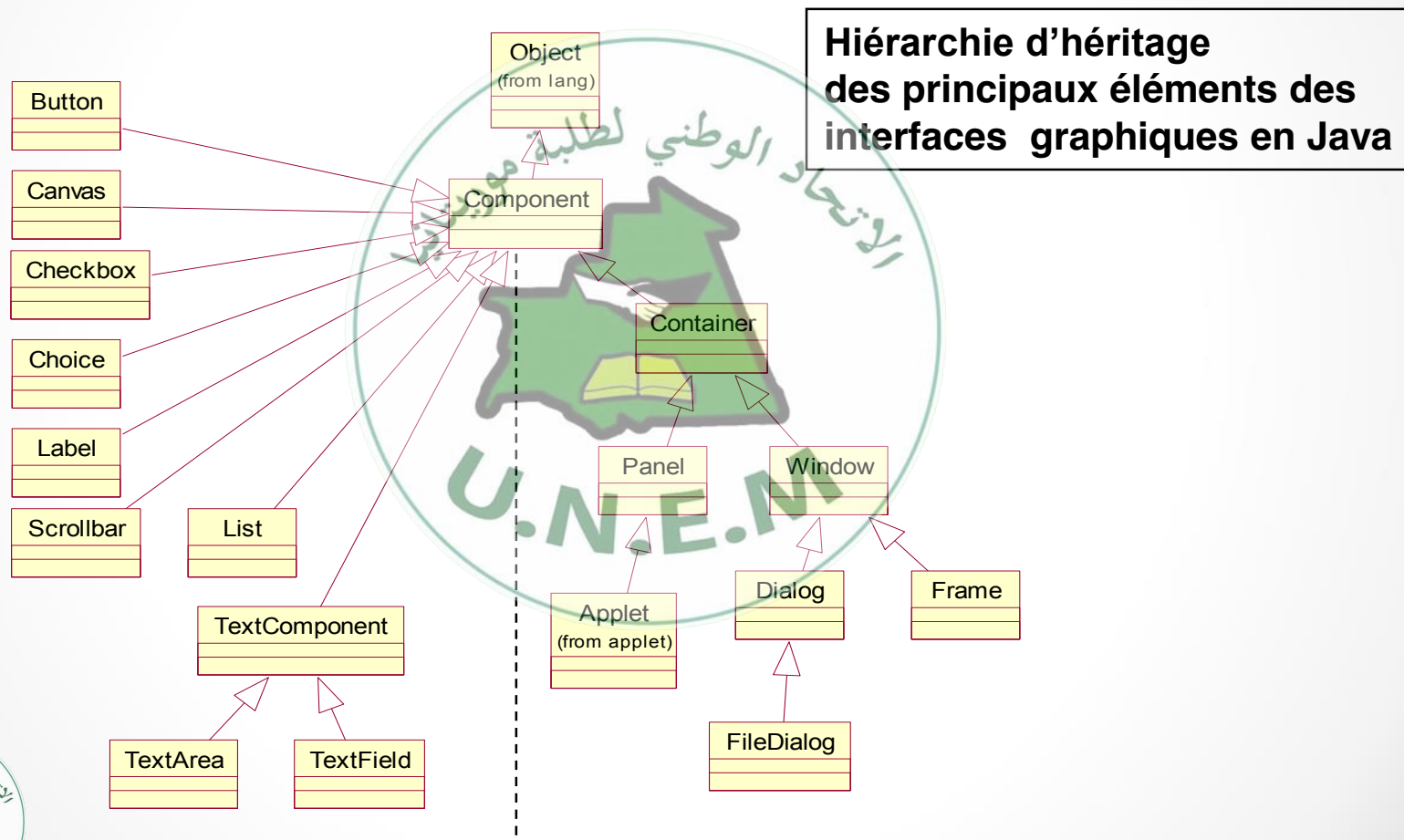


Conteneurs et composants

- Une interface graphique en Java est un assemblage conteneurs (*Container*) et de composants (*Component*).
- **Un composant** est une partie "visible" de l'interface utilisateur Java.
 - C'est une sous-classes de la classe abstraite **java.awt.Component**.
 - Exemple : les boutons, les zones de textes ou de dessin, etc.
- **Un conteneur** est un espace dans lequel on peut positionner plusieurs composants.
 - Sous-classe de la classe **java.awt.Container**
 - La classe Container est elle-même une sous-classe de la classe Component
 - Par exemple les fenêtres, les applets, etc.



Conteneurs et composants



Les Composants Swing

- Menus, Bar d'outils et ToolTips

- JMenuBar

- JMenu

- JMenuItem

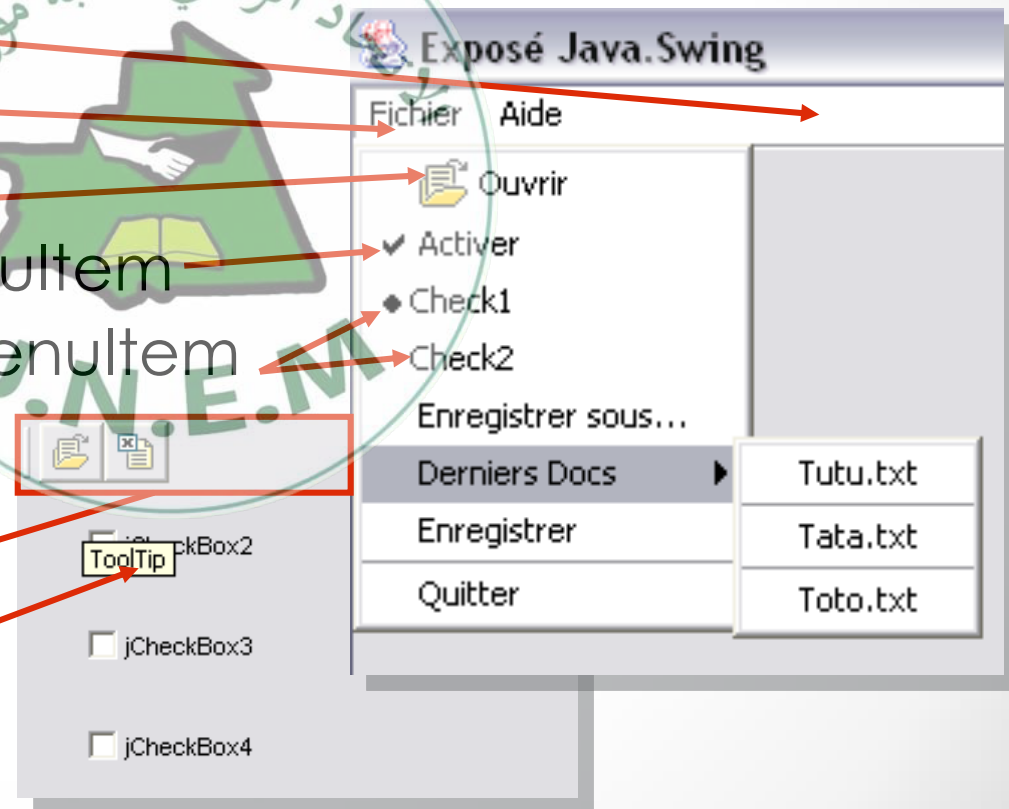
- JCheckBoxMenuItem

- JRadioButtonMenuItem

- JPopupMenu

- JToolBar

- JToolTip



Conteneurs et composants

- Les deux conteneurs les plus courants sont le Frame et le Panel.
- Un **Frame** représente une fenêtre de haut niveau avec un titre, une bordure et des angles de redimensionnement.
 - La plupart des applications utilisent au moins un Frame comme point de départ de leur interface graphique.
- Un **Panel** n'a pas une apparence propre et ne peut pas être utilisé comme fenêtre autonome.
 - Les Panels sont créés et ajoutés aux autres conteneurs de la même façon que les composants tels que les boutons
 - Les Panels peuvent ensuite redéfinir une présentation qui leur soit propre pour contenir eux-mêmes d'autres composants.



Conteneurs et composants

- On ajoute un composant dans un conteneur, avec la méthode `add()` :
`Panel p = new Panel();`
`Button b = new Button();`
`p.add(b);`
- De manière similaire, un composant est retiré de son conteneur par la méthode `remove()` :
`p.remove(b);`
- Un composant a (notamment) :
 - une taille préférée que l'on obtient avec `getPreferredSize()`
 - une taille minimum que l'on obtient avec `getMinimunSize()`
 - une taille maximum que l'on obtient avec `getMaximunSize()`



Conteneurs et composants

Exemple

```
import java.awt.*;  
  
public class EssaiFenetre1  
{  
    public static void main(String[] args)  
    {  
        Frame f =new Frame("Ma première fenêtre");  
        Button b= new Button("coucou");  
        f.add(b);  
        f.pack();  
        f.show();  
    }  
}
```



Création d'une fenêtre
(un objet de la classe
Frame) avec un titre

Création du bouton ayant
pour label « coucou »

Ajout du bouton dans la
fenêtre

On demande à la fenêtre de
choisir la taille minimum avec
pack() et de se rendre visible
avec show()



Gestionnaire de présentation

- A chaque conteneur est associé un gestionnaire de présentation (**layout manager**)
- Le gestionnaire de présentation gère le positionnement et le (re)dimensionnement des composants d'un conteneur.
- Le ré-agencement des composants dans un conteneur a lieu lors de :
 - la modification de sa taille,
 - le changement de la taille ou le déplacement d'un des composants.
 - l'ajout, l'affichage, la suppression ou le masquage d'un composant.
- Les principaux gestionnaires de présentation de l'AWT sont : **FlowLayout, BorderLayout, GridLayout, CardLayout, GridBagLayout**



Gestionnaire de présentation

- Tout conteneur possède un gestionnaire de présentation par défaut.
 - Tout instance de **Container** référence une instance de **LayoutManager**.
 - Il est possible d'en changer grâce à la méthode **setLayout()**.
- Les gestionnaires de présentation par défaut sont :
 - Le **BorderLayout** pour **Window** et ses descendants (**Frame**, **Dialog**, ...)
 - Le **FlowLayout** pour **Panel** et ses descendants (**Applet**, etc.)
- Une fois installé, un gestionnaire de présentation fonctionne "tout seul" en interagissant avec le conteneur.



FlowLayout

- Le **FlowLayout** est le plus simple des managers de l'AWT
-
- Gestionnaire de présentation utilisé par défaut dans les Panel si aucun LayoutManager n'est spécifié.
- Un FlowLayout peut spécifier :
 - une justification à gauche, à droite ou centrée,
 - un espacement horizontal ou vertical entre deux composants.
 - Par défaut, les composants sont centrés à l'intérieur de zone qui leur est allouée.

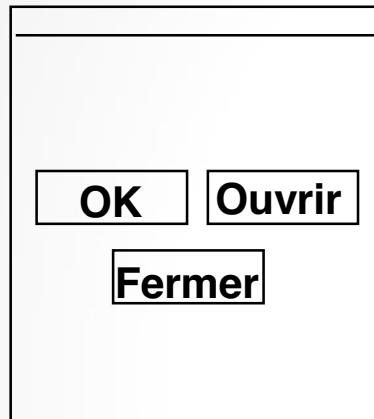


FlowLayout

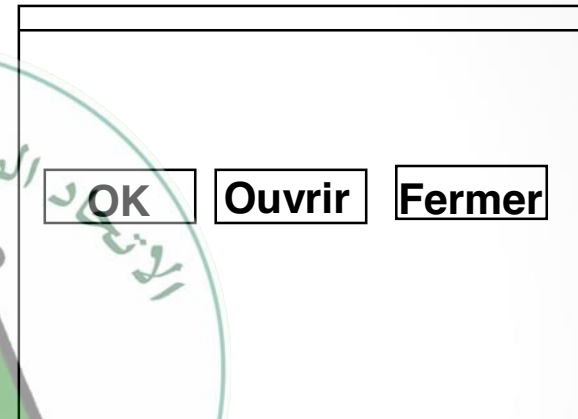
- La stratégie de disposition du **FlowLayout** est la suivante :
 - Respecter la **taille préférée** de tous les composants contenus.
 - Disposer autant de composants que l'on peut en faire tenir horizontalement à l'intérieur de l'objet **Container**.
 - Commencer une nouvelle rangée de composants si on ne peut pas les faire tenir sur une seule rangée.
 - Si tous les composants ne peuvent pas tenir dans l'objet **Container**, ce n'est pas géré (c'est-à-dire que les composants peuvent ne pas apparaître).



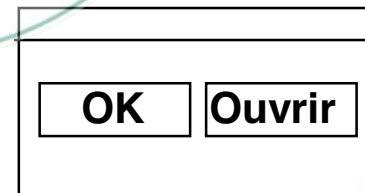
FlowLayout



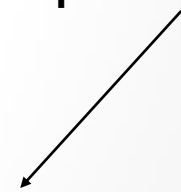
Redimensionnement



Redimensionnement



plus visible



Fermer



FlowLayout



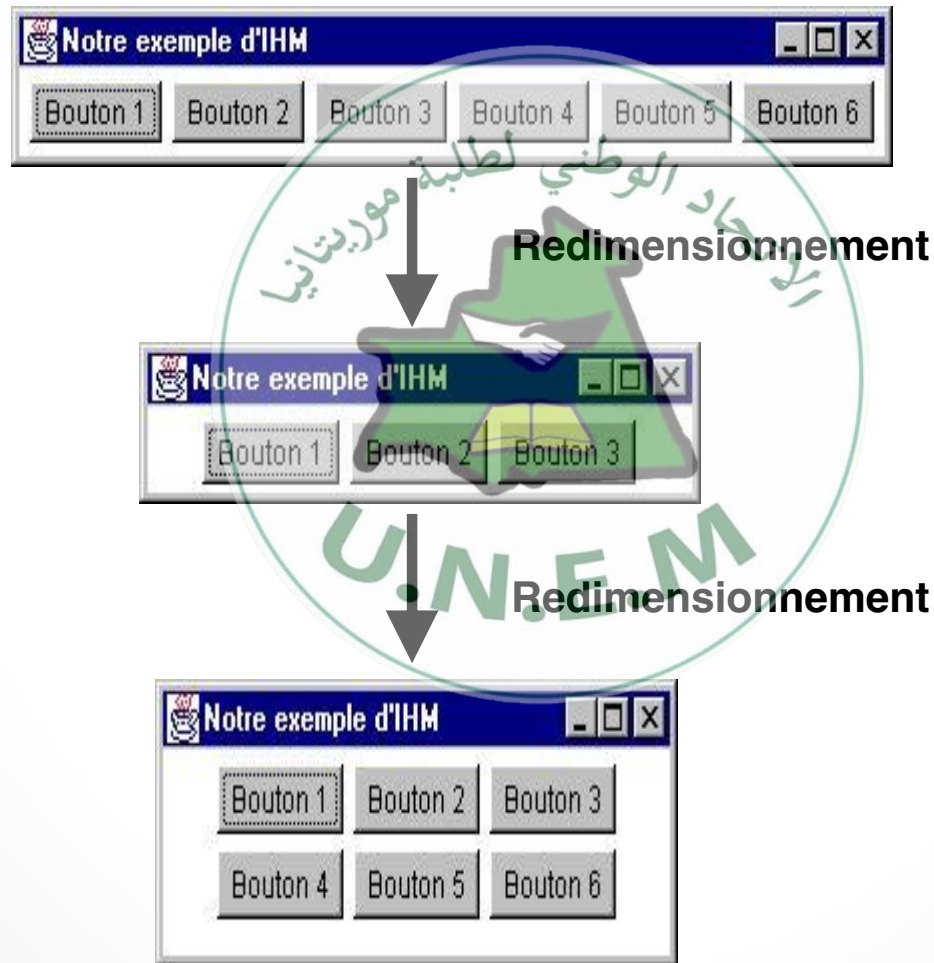
Redimensionnement



Redimensionnement



FlowLayout



FlowLayout

- Le FlowLayout cache réellement et effectivement les composants qui ne rentrent pas dans le cadre.
- Le FlowLayout n'a d'intérêt que quand il y a peu de composants.
- L'équivalent vertical du FlowLayout n'existe pas
- La présentation FlowLayout positionne les composants ligne par ligne.
 - **Chaque fois qu'une ligne est remplie, une nouvelle ligne est commencée.**
 - **Ces composants sont alignés de gauche à droite**
- Le gestionnaire FlowLayout n'impose pas la taille des composants mais leur permet d'avoir la taille qu'ils préfèrent.



FlowLayout

La méthode **setLayout** permet de définir un gestionnaire de positionnement qui se chargera de placer correctement les composants insérés.

Exemple

```
import java.awt.*;  
class FFlowLayout {  
    static public void main (String arg [ ]) {  
        Frame w = new Frame("Exemple de fenetre avec un FlowLayout");  
        w.setLayout(new FlowLayout ());  
        w.add(new Button ("UN"));  
        w.add(new Label ("DEUX"));  
        w.add(new TextField("Trois"));  
        w.show();  
        w.pack();  
    }  
}
```

Résultat à l'exécution :



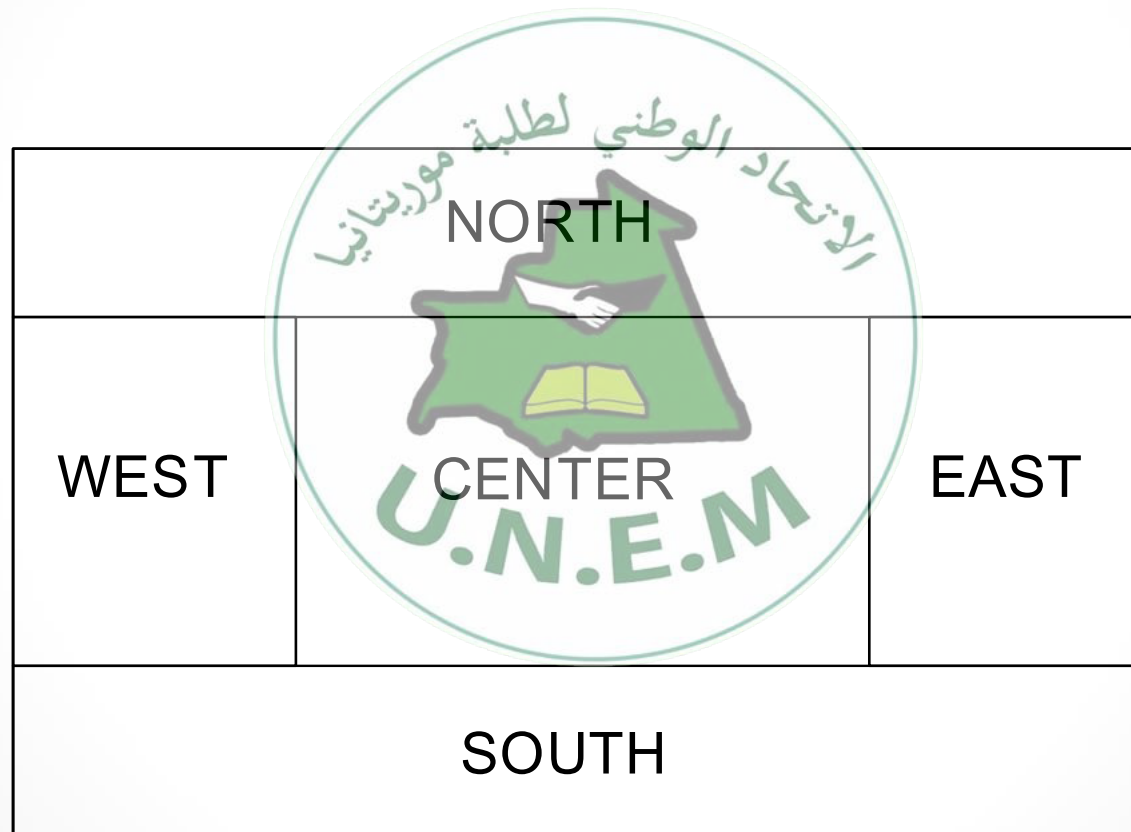
BorderLayout

- **BorderLayout** divise son espace de travail en cinq zones géographiques : **North, South, East, West et Center**.
- Les composants sont ajoutés par nom à ces zones (un seul composant par zone).
 - Exemple
add("North", new Button("Le bouton nord !"));
 - Si une des zones de bordure ne contient rien, sa taille est 0.



BorderLayout

- Division de l'espace avec le **BorderLayout**

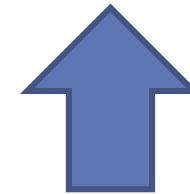


Exemple

BorderLayout

```
import java.awt.*;  
  
public class EssaiBorderLayout extends Frame  
{  
    private Button b1,b2,b3,b4, b5;  
    public EssaiBorderLayout() {  
        setLayout(new BorderLayout());  
        b1 = new Button ("Nord");  
        b2 = new Button ("Sud");  
        b3 = new Button ("Est");  
        b4 = new Button ("Ouest");  
        b5 = new Button ("Centre");  
        this.add(b1, BorderLayout.NORTH);  
        this.add(b2 , BorderLayout.SOUTH);  
        this.add(b3, BorderLayout.EAST);  
        this.add(b4, BorderLayout.WEST);  
        this.add(b5, BorderLayout.CENTER);  
    }  
}
```

Résultat à l'exécution :



```
public static void main (String args []) {  
    EssaiBorderLayout Test = new  
    EssaiBorderLayout();  
    Test.pack ();  
    Test.setVisible(true) ;    }  
}
```



BorderLayout

- Stratégie de disposition du BorderLayout



- S'il y a un composant dans la partie placée dans la partie **NORTH**, il récupère sa **taille préférée**, respecte sa **hauteur préférée** si possible et fixe sa largeur à la totalité de la largeur disponible de l'objet Container.
- S'il y a un composant dans la partie placée dans la partie **SOUTH**, il fait pareil que dans le cas de la partie **NORTH**.
- S'il y a un composant dans la partie placée dans la partie **EAST**, il récupère sa **taille préférée**, respecte sa **largeur préférée** si possible et fixe sa hauteur à la totalité de la hauteur encore disponible.
- S'il y a un composant dans la partie placée dans la partie **WEST**, il fait pareil que dans le cas de la partie **EAST**.



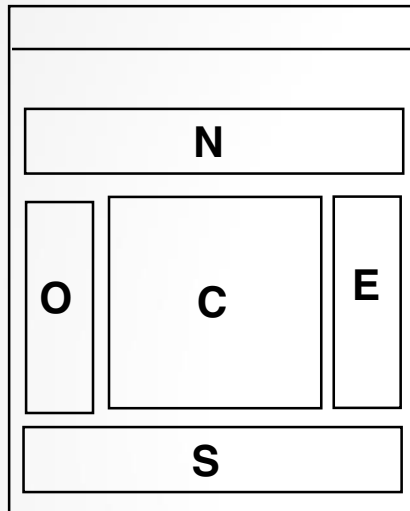
○ S'il y a un composant dans la partie **CENTER**, il lui donne la place qui reste, s'il en reste encore.

BorderLayout

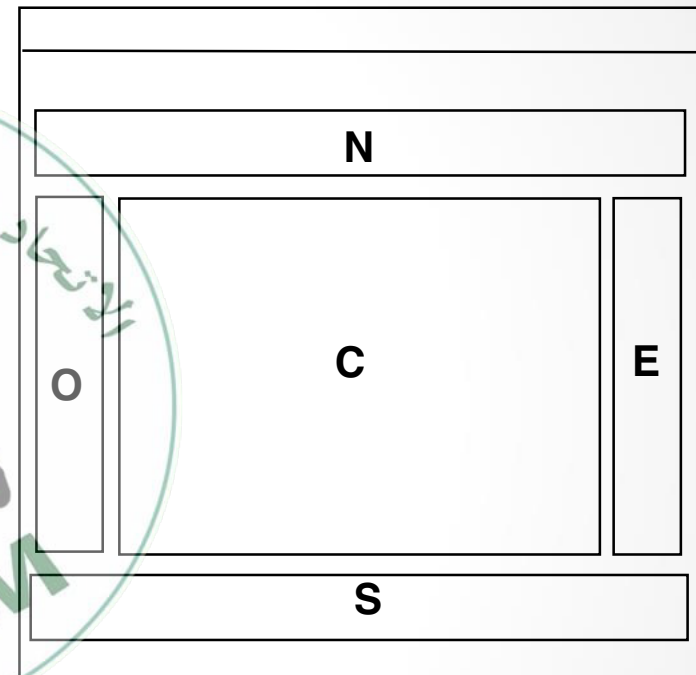
- Lors du redimensionnement, le composant est lui-même redimensionné en fonction de la taille de la zone, c-à-d :
 - les zones nord et sud sont éventuellement **élargies** mais pas allongées.
 - les zones est et ouest sont éventuellement **allongées** mais pas élargies,
 - la zone centrale est **étirée dans les deux sens**.



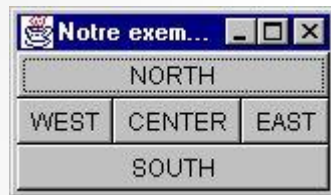
BorderLayout



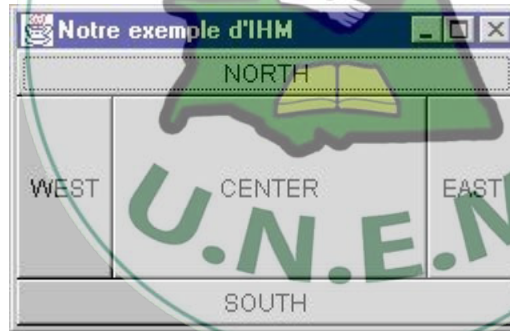
Redimensionnement



BorderLayout



Redimensionnement



Redimensionnement



GridLayout

- Le **GridLayout** dispose les composants dans une grille.
 - Découpage de la zone d'affichage en lignes et en colonnes qui définissent des cellules de dimensions égales.
 - Chaque composant à la même taille
 - quand ils sont ajoutés dans les cellules le remplissage s'effectue de gauche à droite et de haut en bas.
 - Les 2 paramètres sont les rangées et les colonnes.
 - Construction d'un **GridLayout** : **new GridLayout(3,2);**



nombre de lignes

nombre de colonnes

GridLayout

Exemple

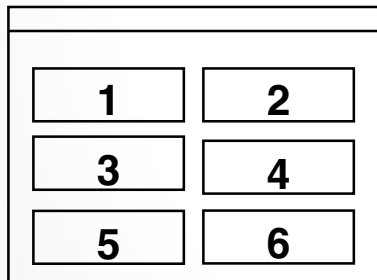
```
import java.awt.*;  
  
public class AppliGridLayout extends Frame  
{  
    public AppliGridLayout()  
    {  
        super("AppliGridLayout");  
        this.setLayout(new GridLayout(3,2));  
        for (int i = 1; i < 7; i++)  
            add(new Button(Integer.toString(i)));  
        this.pack();  
        this.show();  
    }  
  
    public static void main(String args[])  
    {  
        AppliGridLayout appli = new AppliGridLayout();  
    }  
}
```



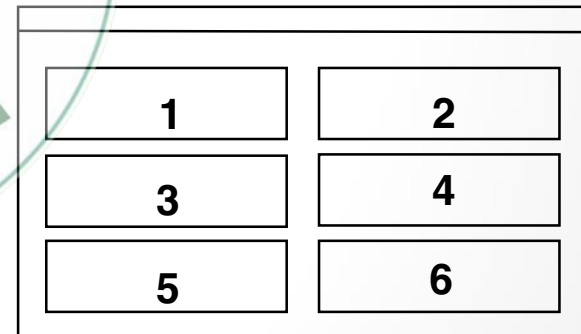
Résultat à l'exécution :

GridLayout

- Lors d'un redimensionnement les composants changent tous de taille mais leurs positions relatives ne changent pas.



Redimensionnement

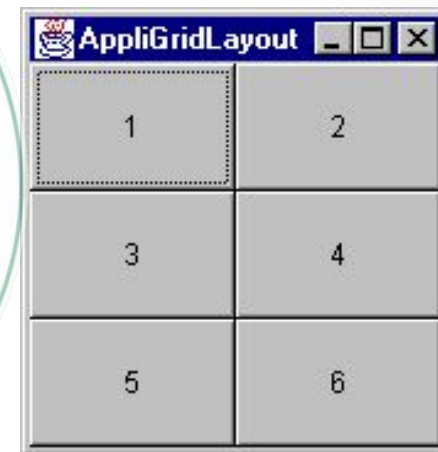


GridLayout



1	2
3	4
5	6

Redimensionnement



1	2
3	4
5	6



CardLayout

- Le **CardLayout** n'affiche qu'un composant à la fois :
 - les composants sont considérées comme empilées, à la façon d'un tas de cartes.
- La présentation **CardLayout** permet à plusieurs composants de partager le même espace d'affichage de telle sorte que seul l'un d'entre eux soit visible à la fois.
- Pour ajouter un composant à un conteneur utilisant un CardLayout il faut utiliser `add(String cle, Component monComposant)`
- Permet de passer de l'affichage d'un composant à un autre en appelant les méthodes **first**, **last**, **next**, **previous** ou



GridBagLayout

- Le gestionnaire **GridBagLayout** fournit des fonctions de présentation complexes
 - basées sur une grille dont les lignes et les colonnes sont de taille variables.
 - permet à des composants simples de prendre leur taille préférée au sein d'une cellule, au lieu de remplir toute la cellule.
 - permet aussi l'extension d'un même composant sur plusieurs cellules.
- Le GridBagLayout est compliqué à gérer.
 - Dans la plupart des cas, il est possible d'éviter de l'utiliser en associant des objets **Container** utilisant des gestionnaires différents.



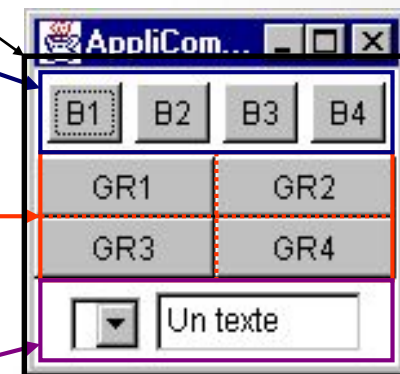
GridBagLayout

- Le gestionnaire **GridBagLayout** est associé à un objet **GridBagConstraints**
 - l'objet **GridBagConstraints** définit des contraintes de positionnement, d'alignements, de taille, etc. d'un composant dans un conteneur géré par un **GridBagLayout**.
 - On associe chaque composant que l'on place dans le **GridBagLayout** avec un objet **GridBagConstraints**
 - Un même objet **GridBagConstraints** peut-être associé à plusieurs composants.
 - Définir les objets **GridBagConstraints** en spécifiant les différents paramètres est assez fastidieux...
 - Voir la doc



Mise en forme complexe

```
super("AppliComplexeLayout");  
setLayout(new BorderLayout());  
Panel pnorth = new Panel();  
pnorth.add(b1); pnorth.add(b2);  
pnorth.add(b3); pnorth.add(b4);  
this.add(pnorth, BorderLayout.NORTH);  
  
Panel pcenter = new Panel();  
pcenter.setLayout(new GridLayout(2,2));  
pcenter.add(gr1); pcenter.add(gr2);  
pcenter.add(gr3); pcenter.add(gr4);  
this.add(pcenter, BorderLayout.CENTER);  
  
Panel psouth = new Panel();  
psouth.setLayout(new FlowLayout());  
psouth.add(ch); psouth.add(tf);  
this.add(psouth, BorderLayout.SOUTH);
```



D'autres gestionnaires?

- On peut imposer à un objet « container » de n'avoir pas de gestionnaire en fixant son **LayoutManager** à la valeur null
 - **Frame f = new Frame(); f.setLayout(null);**
 - A la charge alors du programmeur de positionner chacun des composants « manuellement » en indiquant leur position absolue dans le repère de la fenêtre.
 - C'est à éviter, sauf dans des cas particuliers,.
- Il est possible d'écrire ses propres **LayoutManager...**



Récapitulatif

- **FlowLayout**

- Flux : composants placés les uns derrière les autres

- **BorderLayout**

- Ecran découpé en 5 zones (« North », « West », « South », « East », « Center »)

- **GridLayout**

- Grille : une case par composant, chaque case de la même taille

- **CardLayout**

- « Onglets » : on affiche un élément à la fois

- **GridBagLayout**



- Grille complexe : plusieurs cases par composant